



分布与并行数据库

3 分布式数据库设计

主讲教师：鄂金龙
中国人民大学信息学院



内 容 简 介

- 在系统设计中，最基本问题就是数据的分布问题，即如何对分布式数据库进行逻辑划分和实际物理分配。数据的逻辑划分称数据分片。本章主要针对数据分布进行介绍，以关系数据库为例来说明。同时介绍当前流行的大数据库的分布存储策略。
 - 两种设计策略（ Top-Down 和 Bottom-Up ）
 - 分片的定义
 - 水平分片
 - 垂直分片
 - 混合分片
 - 分片的表示方法
 - 分配设计
 - 数据复制技术
 - 大数据库的分布存储策略

两种设计策略

□ **Top-Down**和 **Bottom-UP**应用场合

- **Top-Down:** 适于新设计一个数据库系统。
(**designed from scratch**)
- **Bottom-UP:** 已存在许多数据库系统，并将它们集成为一个数据库。典型的数据集成研究。

本课程侧重 **Top-Down** 设计中的关键技术。

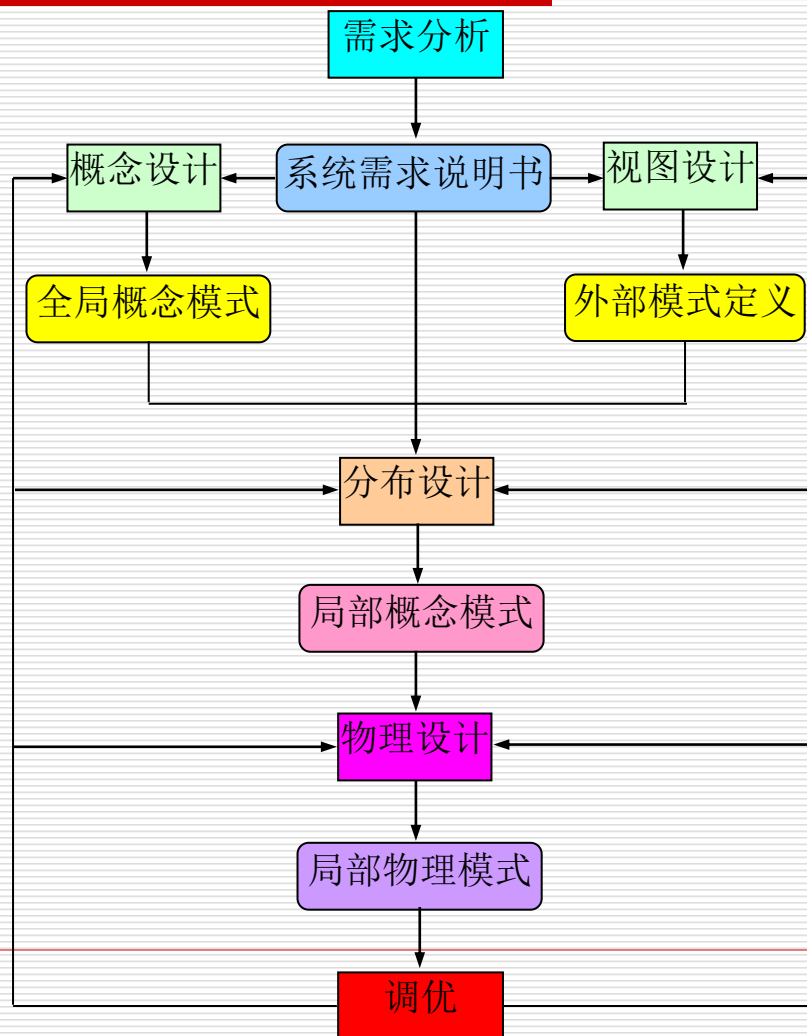
两种设计策略

□ Top-Down 设计过程

基于五步递归实现：

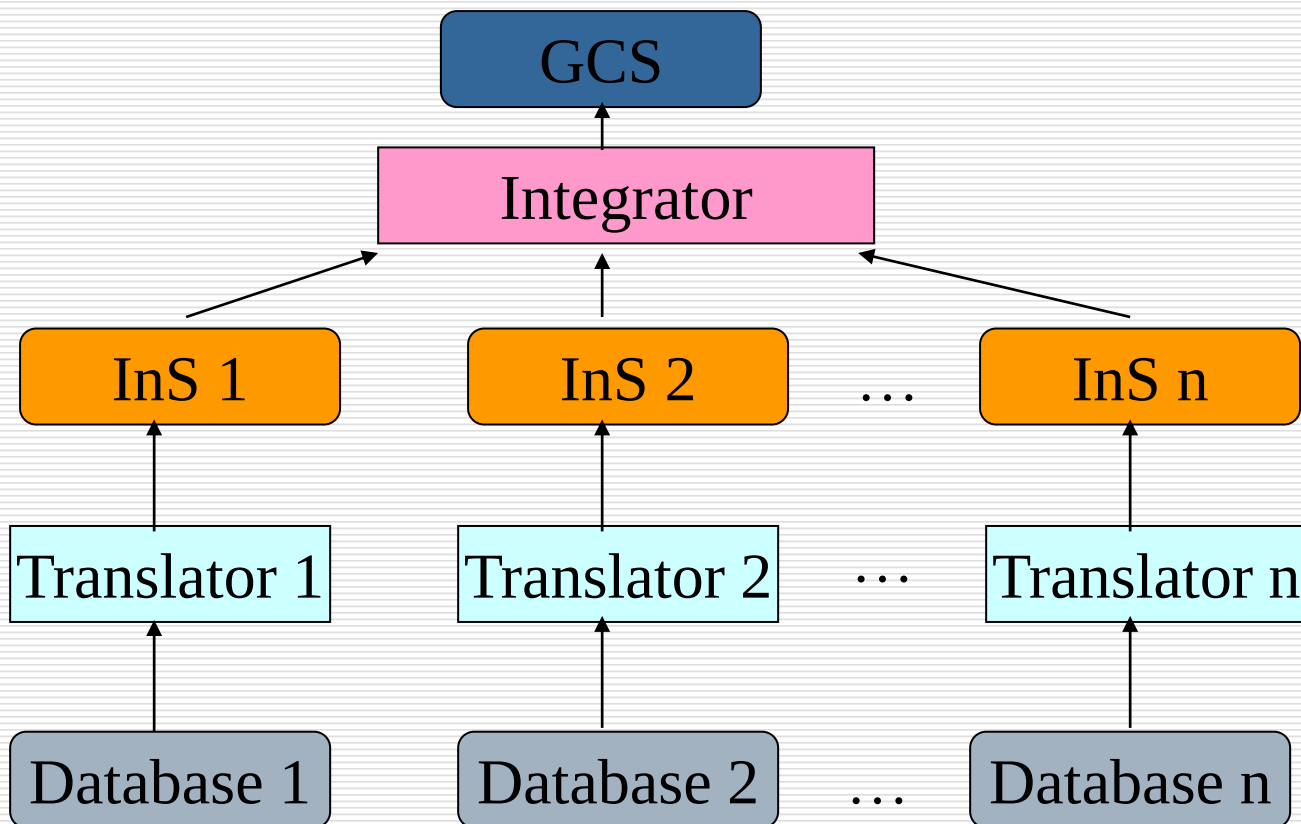
1. 需求分析 (**Requirement analysis**)
2. 概念设计、视图集成、**E-R** 表示、转换到关系模式
3. 分布设计、数据分片和分配
4. 物理设计
5. 性能调优 (**Tuning**)

两种设计策略：Top-Down



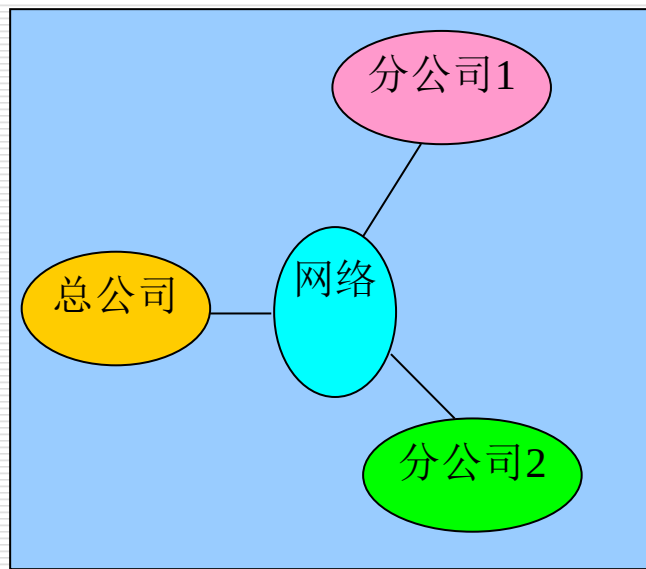
两种设计策略: Bottom-Up

□ 源于多数据库应用



分片的定义

- 例：某集团公司由地理位置分别在不同城市的总公司和两个下属分公司组成，彼此之间靠网络相连接，业务管理由分布式数据库系统完成。其网络结构图如图所示：



假设：人事系统中，有职工关系，定义为：
 $EMP \{ENO, EName, Salary, DNO\}$ 。

场地定义：总公司为场地0，职工关系为
EMP0

分公司1为场地1，职工关系为EMP1

分公司2为场地2，职工关系为EMP2

$EMP = EMP0 + EMP1 + EMP2$ 为全局数据

分片的定义

□ 数据分布要求

方案1：公司总部保留全部数据。

方案2：各单位只保留自己的数据。

方案3：总公司保留全部数据，各分公司只保留自己单位的数据。

系统采用以上不同方案，对应需求不同的数据分配方案。如：

	方案1	方案2	方案3
场地0	总部（EMP）	总部（EMP0）	总部（EMP）
场地1	分公司1（无）	分公司1（EMP1）	分公司1（EMP1）
场地2	分公司2（无）	分公司2（EMP2）	分公司2（EMP2）

□ 上面三种方案中，除方案1外，均须进行分片定义。方案3中，分公司的数据信息除本场地存储外，总部场地也存储一份相同信息，存在在不同场地上互称副本。

分片的定义

□ 定义

- **片段 (Fragment)**：是分布式数据库中数据的存储单位。
- **分片 (Fragmentation)**：对全局数据的划分。
- **片段**：划分的结果称为**片段**。
- **分配 (Allocation)**：对片段存储场地的指定，称为**分配**。
当片段存储在一个以上场地时，称为**数据复制 (Replication)**存储。如果每个片段只存储在一个场地，称为**数据分割 (Partition)**存储。

分片的定义

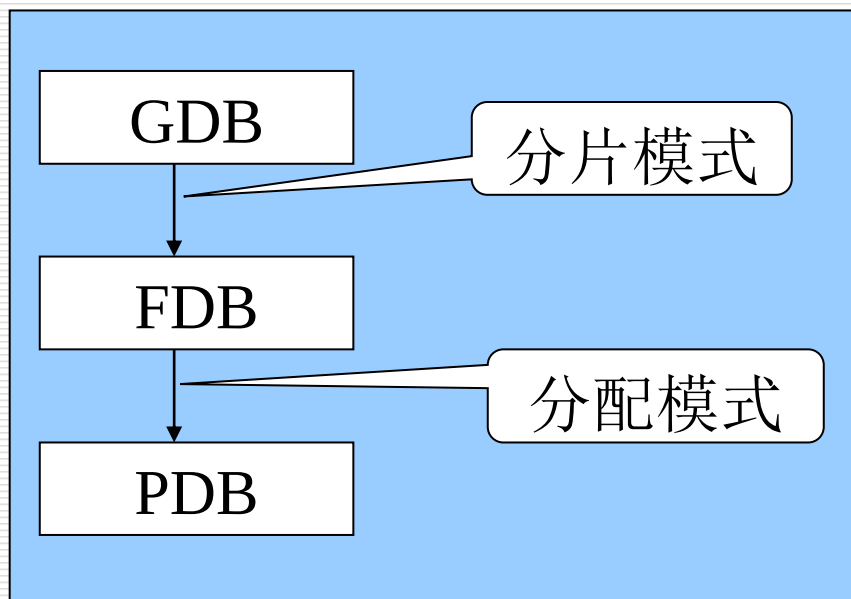
□ 分片的作用

- **减少网络传输量。**如：采用数据复制，可就近访问所需信息。需频繁访问的数据分片存储在本地场地上。
- **增大事务处理的局部性。**局部场地所需的数据分片分配在各自的场地上，减少数据访问的时间，增强局部事务效率。
- **提高数据的可用性和查询效率。**就近访问数据分片或副本，可提高数据的访问效率。同时当某一场地发生故障时，若存在副本，非故障场地上的数据副本均是可用的。保证了数据的可用性、数据的完整性和系统的可靠性。
- **使负载均衡。**减少数据访问瓶颈，提高整个系统效率。

分片的定义

□ 分片设计过程

分片过程是将全局数据进行逻辑划分和实际物理分配的过程。全局数据由**分片模式**定义分成各个片段数据，各个片段数据由**分配模式**定义存储在各场地上。分片过程如下图所示。



GDB : 全局数据库
(Global DB)

FDB : 片段数据库
(Fragmentation DB)

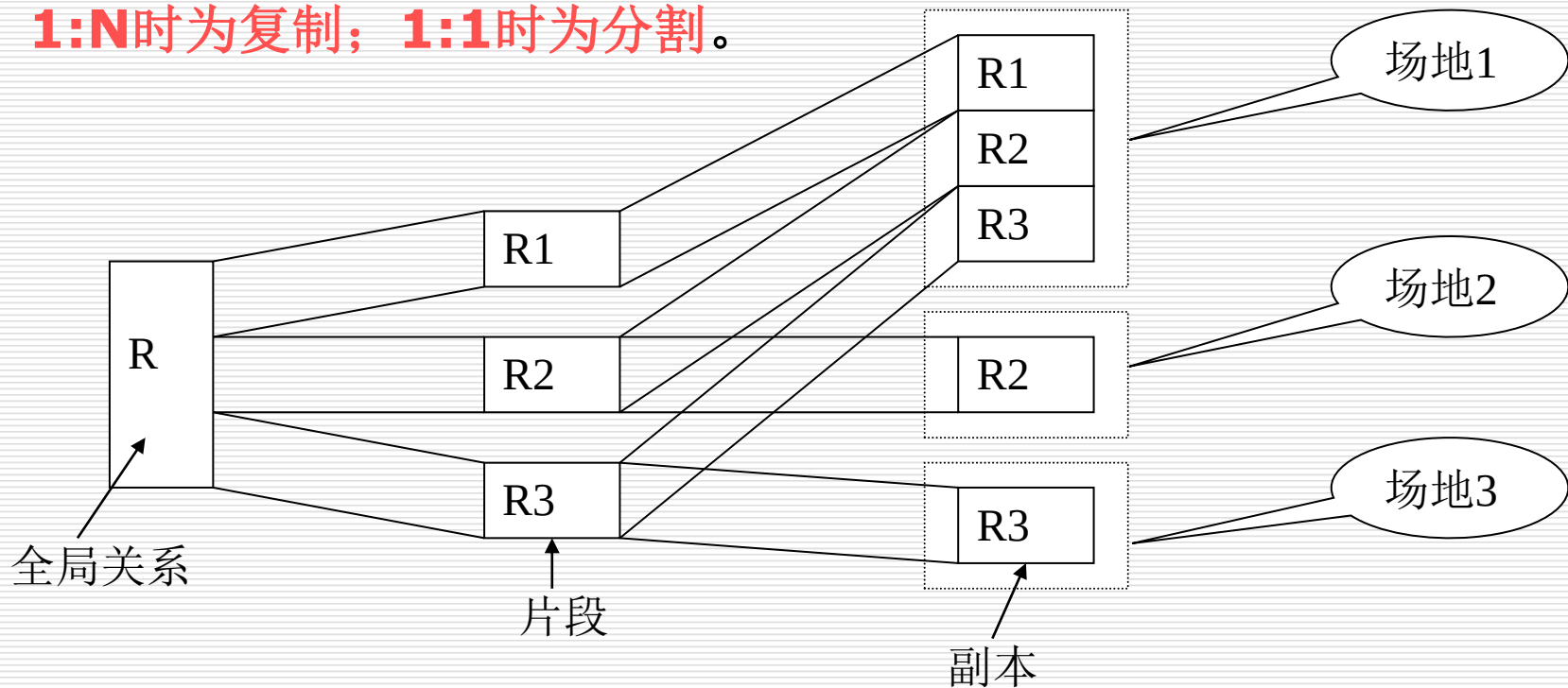
PDB : 物理数据库
(Physical DB)

分片的定义

□ **分片模式**：定义从全局模式到片段模式的映射关系。

□ **分配模式**：定义从片段模式到物理模式的映射关系。

1:N时为复制；**1:1**时为分割。



分片的定义

□ 分片设计过程说明

$$GDB = \sum FDB_i$$

表示全局数据库是经划分的多个片段数据库的集合。

$$F(GDB) = FDB \iff F^{-1}(FDB) = GDB$$

$F()$: 表示分片模式映射函数, $F^{-1}()$ 表示分片模式的反映射函数。即全局数据库经分片模式映射函数得到全局数据的各个片段; 相反, 所有片段经分片模式反映射函数得到全局数据。

$$P(FDB) = PDB \iff P^{-1}(PDB) = FDB$$

$P()$: 表示分配模式映射函数, $P^{-1}()$ 表示分配模式的反映射函数。即片段数据库经分配模式映射函数得到物理数据库; 相反, 物理数据库经分配模式反映射函数得到片段数据库。

分片的定义

□ 分片的原则

在构成分布式数据库系统时，设计者必须考虑数据如何分布在各个场地上，即全局数据如何进行逻辑划分和物理分配问题。**哪些数据须分布存放、哪些数据不需要分布存放、哪些数据需要复制等**，对系统进行全盘考虑，使系统性能最优。但无论如何进行分片，必须遵循下面原则：

- **完备性**：所有全局数据必须映射到某个片段上。
- **可重构性**：所有片段必须可以重新构成全局数据。
- **不相交性**：划分的各片段所包含的数据不允许相同。

分片的定义

□ 分片的原則

■ 完备性

如果全局关系 R 划分的片段为 $R_1, R_2, \dots, R_n$ ，则对于 R 中任意数据项 $d(d \in R)$ ，一定存在 $d \in R_i (1 \leq i \leq n)$ 。

■ 可重构性

如果全局关系 R 划分的片段为 $R_1, R_2, \dots, R_n$ ，则存在关系运算 ψ ，使得 $R = R_1 \psi R_2 \psi \dots \psi R_n$ 。

■ 不相交性

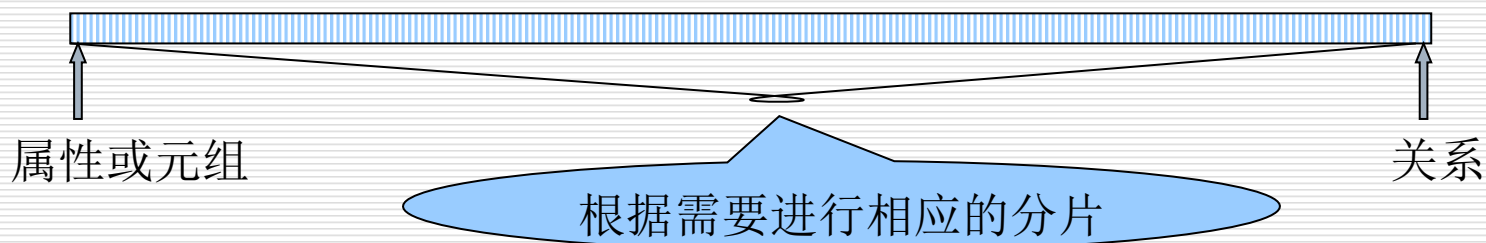
如果全局关系 R 水平划分的片段为 $R_1, R_2, \dots, R_n$ ，则任意两个不同片段的交集为空，即 $R_i \cap R_j = \phi (i \neq j, 1 \leq i, j \leq n)$ 。

分片的定义

□ 分片的种类

分布式系统按系统实际需求对全局数据进行分片和物理分配。分片的种类有三种。

- **水平分片**：按元组进行划分，由分片条件决定。
- **垂直分片**：按关系属性划分，除关键字外，其他属性不允许出现在多个分片中。
- **混合分片**：即包括水平分片也包括垂直分片。



分片的定义

□ 分布透明性

- **透明性**：指对用户和高层系统隐蔽具体实现细节。

包括：分片透明性、分配透明性和局部映射透明性。

- **分片透明性**：用户不必考虑数据属于哪个片段。

- **分配透明性**：用户不必考虑各个片段的存储场地。

- **局部映射透明性**：用户不必考虑数据的局部存储形式。

水平分片

□ 水平分片的定义

水平分片是将关系的元组集划分成若干不相交的子集。每个水平片段由关系中的某个属性上的条件来定义，该属性称为**分片属性**，该条件称为**分片条件**。

■ 定义

设有一个关系 R ， $\{R_1, R_2, \dots, R_n\}$ 为 R 的子关系的集合，如果 $\{R_1, R_2, \dots, R_n\}$ 满足以下条件，则称其为关系 R 的水平分片， R_i 称为 R 的一个水平片段。

(1) $R_1, R_2, \dots, R_n$ 与 R 具有相同的模式

(2) $R_1 \cup R_2 \cup \dots \cup R_n = R$

(3) $R_i \cap R_j = \Phi \ (i \neq j, 1 \leq i, j \leq n)$

水平分片

□例如：设有雇员关系 **EMP{ENO, ENAME, SALARY, DNO}**

ENO	ENAME	SALARY	DNO
001	张三	1500	201
002	李四	1400	202
003	王五	800	203

按下面分片条件进行分片：

E1：满足（**Dno=201**）的所有分组；

E2：满足（**Dno=202**）的所有分组；

E3：满足（**Dno <> 201 AND Dno <> 202**）
的所有分组。

水平分片

□ 将关系 **EMP** 分成了三个子关系，部门编号 **Dno** 等于 **201** 的元组 (**E1**)、部门编号 **Dno** 等于 **202** 的元组 (**E2**) 和其他元组 (**E3**)。

分片属性：部门编号 **Dno**

分片条件：

E1: Dno=201

E2: Dno=202

**E3: Dno <> 201 AND
Dno <> 202**

ENO	ENAME	SALARY	DNO
001	张三	1500	201
002	李四	1400	202
003	王五	800	203

E1:	001	张三	1500	201
E2:	002	李四	1400	202
E3:	003	王五	1200	203

水平分片

根据水平分片定义，满足：

- $E1, E2, E3$ 和 EMP 关系模式相同；
- $E1 \cup E2 \cup E3 = EMP$
- $E1 \cap E2 = \Phi$, $E1 \cap E3 = \Phi$, $E2 \cap E3 = \Phi$

因此， $E1$ 、 $E2$ 和 $E3$ 是 EMP 的水平分片。

水平分片

■ 水平分片的表示

水平分片实际上是关系的选择操作，因此，用选择操作 σ 表示，选择条件为分片谓词 q ，则 R 的片段可表示为： $\sigma_q(R)$ 。

例： $E1 = \sigma_{Dno=201}(EMP)$

SQL: SELECT * FROM EMP WHERE DNO=201

$E2 = \sigma_{Dno=202}(EMP)$

SQL: SELECT * FROM EMP WHERE DNO=202

$E3 = \sigma_{Dno \neq 201 \text{ AND } Dno \neq 202}(EMP)$

SQL: SELECT * FROM EMP WHERE Dno $\neq$ 201
AND Dno $\neq$ 202

水平分片

■ 正确性判断

分片必须遵循完备性、可重构性和不相交性三个原则。通过验证满足这三个特性判断分片的正确性。例如，上例的验证过程：

□ 完备性证明：

(求证 $(Dno=201) \cup (Dno=202) \cup (Dno \neq 201 \cap Dno \neq 202) = T$)

证明：

$(Dno=201) \cup (Dno=202) \cup (Dno \neq 201 \cap Dno \neq 202)$

$= ((Dno=201) \cup (Dno=202)) \cup (\neg ((Dno=201) \cup (Dno=202)))$

$= T$

满足完备性。

水平分片

■ 正确性判断

□ 可重构性证明

(求证 $E1 \cup E2 \cup E3 = EMP$)

证明:

$E1 \cup E2 \cup E3$

$= \sigma_{Dno=201}(EMP) \cup \sigma_{Dno=202}(EMP) \cup \sigma_{Dno \neq 201 \text{ AND } Dno \neq 202}(EMP)$

$= \sigma_{Dno=201 \vee Dno=202 \vee (Dno \neq 201 \wedge Dno \neq 202)}(EMP)$

$= \sigma_T(EMP)$

$= EMP$

满足可重构性。

水平分片

■ 正确性判断

□ 不相交性证明

(求证 $E1 \cap E2 = \Phi$, $E1 \cap E3 = \Phi$, $E2 \cap E3 = \Phi$)

证明:

$E1 \cap E2$

$= \sigma_{Dno=201 \wedge Dno=202} (EMP)$

$= \sigma_F (EMP)$

$= \Phi$

同理: $E1 \cap E3 = \Phi$, $E2 \cap E3 = \Phi$

满足不相交性。

根据上面三原则证明可知: 该水平分片的设计是正确的。

垂直分片

□ **垂直分片**是将一个关系按属性集合分成不相交的子集（主关键字除外），属性集合称为分片属性。即垂直分片是将关系按列纵向以属性组划分成若干片段。

□ **定义**：如果R的子关系 $\{R_1, R_2, \dots, R_n\}$ 满足以下条件，则称其为R的垂直分片。

$$(1) \text{Attr}(R_1) \cup \text{Attr}(R_2) \cup \dots \cup \text{Attr}(R_n) = \text{Attr}(R)$$

其中， $\text{Attr}(R)$ 表示关系R的属性集

(2) $\{R_1, R_2, \dots, R_n\}$ 是**关系R的无损分解**。

$$(3) \text{Attr}(R_i) \cap \text{Attr}(R_j) = P_k(R) \quad (i \neq j, \text{ 且 } 1 \leq i, j \leq n)$$

其中， $P_k(R)$ 表示R主关键字

垂直分片

□ 例：一雇员关系

EMP {ENO, ENAME, BIRTH, SALARY, DNO}

ENO, ENAME, BIRTH, SALARY, DNO为雇员编码、
雇员姓名、出生日期、工资、部门编号，其元组为：

ENO	ENAME	BIRTH	SALARY	DNO
001	张三	1960. 5. 2	1500	201
002	李四	1957. 3. 5	1400	202
003	王五	1985. 2. 4	1200	203

垂直分片

- 假设: $E1 \{ ENO, ENAME, BIRTH \}$
 $E2 \{ ENO, SALARY, DNO \}$

则E1和E2元组分别为:

E1元组:		
ENO	ENAME	BIRTH
001	张三	1960. 5. 2
002	李四	1957. 3. 5
003	王五	1985. 2. 4

E2元组:		
ENO	SALARY	DNO
001	1500	201
002	1400	202
003	1200	203

根据垂直分片条件, 可知:

E1和E2是EMP的无损分解。

$$\text{Attr}(E1) \cup \text{Attr}(E2) = \text{Attr}(EMP)$$

$$\text{Attr}(E1) \cap \text{Attr}(E2) = \{ ENO \}$$

因此, E1和E2是EMP的垂直分片。

垂直分片

■ 垂直分片的表示方法

垂直分片是指指定属性集上的投影操作。用投影运算 Π 表示，投影属性为分片属性。如上例E1、E2表示为：

$E1 = \pi_{ENO, ENAME, BIRTH} (EMP)$,

SQL: SELECT ENO, ENAME, BIRTH FROM EMP

$E2 = \pi_{ENO, SALARY, DNO} (EMP)$,

SQL: SELECT ENO, SALARY, DNO FROM EMP

垂直分片

EMP {ENO, ENAME, BIRTH, SALARY, DNO}

$E1 = \pi_{ENO, ENAME, BIRTH} (EMP)$

$E2 = \pi_{ENO, SALARY, DNO} (EMP)$

■ 正确性检验

垂直分片的正确性检验同水平分片正确性检验一样。垂直分片也应满足完备性、可重构性和不相交性。以上例为例。

□ 完备性证明

(求证 $\text{Attr}(E1) \cup \text{Attr}(E2) = \text{Attr}(EMP)$)

证明:

$\{ ENO, ENAME, BIRTH \} \cup \{ ENO, SALARY, DNO \}$

$= \{ ENO, ENAME, BIRTH, SALARY, DNO \}$

满足完备性。

垂直分片

■ 正确性检验

□ 可重构性证明:

目标: 通过关键字将各垂直分片 (E1、E2) 进行连接得到原关系EMP

证明: $E1 \bowtie E2 = EMP$ 等价于如下SQL语句:

SQL:

```
SELECT  E1.ENO, E1.ENAME, E1.BIRTH, E2.SALARY, E2.DNO  
FROM    E1、E2  
WHERE   E1.ENO= E2.ENO
```

可知, 可通过连接操作 $E1 \bowtie E2$ 得到的关系元组同EMP相同。

满足可重构性。

垂直分片

■ 正确性检验

□ 不相交性证明

(求证: $\text{Attr}(E1) \cap \text{Attr}(E2) = P_K(\text{EMP})$)

证明:

$\text{Attr}(E1) \cap \text{Attr}(E2)$

$= \{ \text{ENO}, \text{ENAME}, \text{BIRTH} \} \cap \{ \text{ENO}, \text{SALARY}, \text{DNO} \}$

$= \{ \text{ENO} \}$

$= P_K(\text{EMP})$

因此, 满足不相交性。

- 上述证明, 该垂直分片满足完备性、可重构性和不相交性。所以该分片是正确的。

垂直分片

□ 影响垂直分片设计的因素

- **应用需求**: 同一垂直片段中的多个属性通常是被同一应用同时访问。
- **垂直分片设计的核心**是根据用户的应用需求如何正确地划分属性组。
 - 采用属性紧密度(**affinity**)来度量属性间的关系。
 - 紧密度: 令 $Q = \{q_1, q_2, \dots, q_m\}$ 是用户的查询应用, 关系 $R\{A_1, A_2, \dots, A_n\}$ 包含 $A_1, A_2, \dots, A_n$ 属性, 则 $aff(A_i, A_j)$ 表示属性 A_i, A_j 的紧密度, 描述为:

$$aff(A_i, A_j) = \sum_{l=1}^s \sum_{k=1}^m (ref_l(q_k) \cdot acc_l(q_k))$$

$ref_l(q_k)$ 为查询应用 q_k 在场地 S_l 上同时访问属性 A_i, A_j 的次数

$acc_l(q_k)$ 为查询应用 q_k 在场地 S_l 上的访问频率。

垂直分片

$$aff(A_i, A_j) = \sum_{l=1}^s \sum_{k=1}^m (ref_l(q_k) \cdot acc_l(q_k))$$

□ 举例：存在关系模式：

EMP {ENO, ENAME, SALARY, DNO}, 存在3个场地。

有下面查询应用：

Q1: select SALARY from EMP Where ENO=value

Q2: select ENAME, SALARY from EMP

Q3: select ENAME from EMP Where DNO=value

Q4: select sum(SALARY) from EMP Where DNO=value

	ENO	ENAME	SALARY	DNO
q1	1	0	1	0
q2	0	1	1	0
q3	0	1	0	1
q4	0	0	1	1

	ENO	ENAME	SALARY	DNO
ENO	34	0	34	0
ENAME	0	30	5	40
SALARY	34	5	34	2
DNO	0	45	3	35

令： $ref_1(q_1)=1$ ，假设：
 $acc_1(q_1)=10$, $acc_2(q_1)=15$,
 $acc_3(q_1)=9$,
 则， $aff(ENO, SALARY) = \sum_{k=1}^1 \sum_{l=1}^3 = 10+15+9=34$

混合分片

混合分片是既包括水平分片，又包括垂直分片的分片过程，称为混合分片。下面以一个例子来说明。

例：一雇员关系 **EMP {ENO, ENAME, BIRTH, SALARY, DNO}**

其元组同前

混合分片示意图为：

ENO	ENAME	BIRTH	SALARY	DNO
E1			E21	
			E22	
			E23	

混合分片

先进行垂直分片，分为**E1**和**E2**。然后，将**E2**进行水平分片，分为**E21**、**E22** 和**E23**。

分片表示为：

$$E1 = \pi_{ENO, ENAME, BIRTH} (EMP)$$

$$E2 = \pi_{ENO, SALARY, DNO} (EMP)$$

$$E21 = \sigma_{Dno=201} (E2)$$

$$E22 = \sigma_{Dno=202} (E2)$$

$$E23 = \sigma_{Dno \neq 201 \text{ AND } Dno \neq 202} (E2)$$

分片的表示方法

- 采用直观的图形表示法外，还提供了分片树表示法。

- 图形表示法

用图形直观描述。说明：

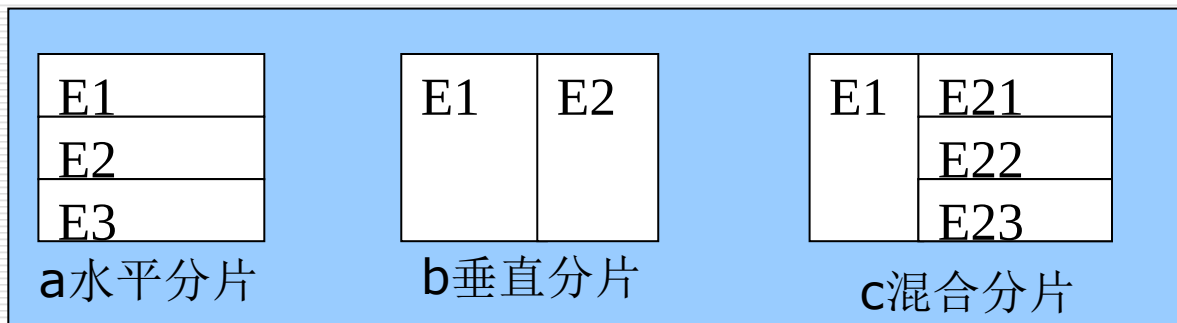
- 整体全部表示为全局关系；
- 部分表示分段；
- 按水平划分表示水平分段；
- 按垂直划分表示垂直分段；
- 混合划分既有水平划分，又有垂直划分。

分片的表示方法

■ 图形表示法

具体图形表示见下图。

- ❑ 关系E水平分片为E1、E2和E3。
- ❑ 关系E垂直分片为E1、E2。
- ❑ 关系E混合分片为E1（垂直分片）和对垂直分片E2的水平分片E21、E22和E23。



分片的表示方法

■ 分片树表示方法

一个分片映射可用分片树表示。先了解分片树的构成。

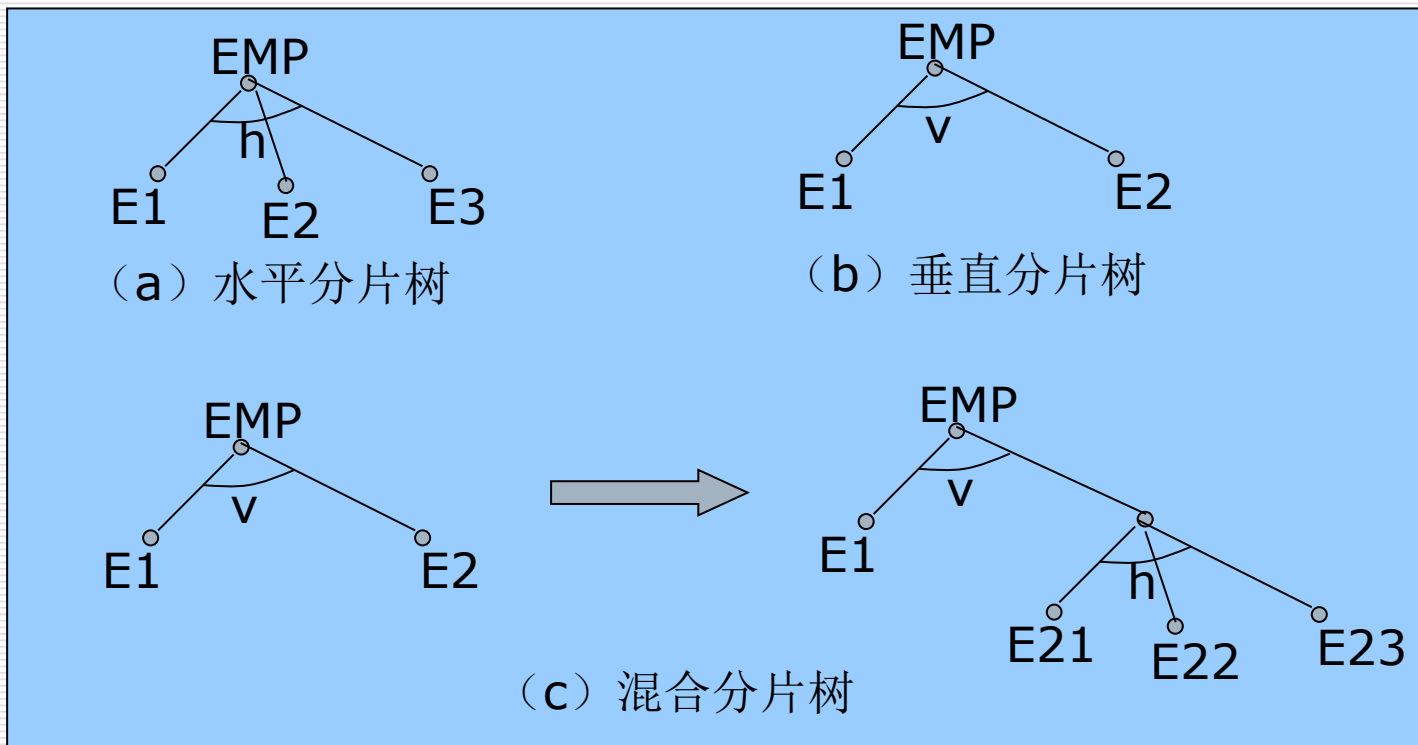
定义：一个分片树由以下几部分组成：

- 根节点表示全局关系；
- 叶子节点表示最后得到的片段；
- 中间节点表示分片过程的中间结果；
- 边表示分片，并用**h**（水平）和**v**（垂直）表示分片类型
- 节点名表示全局关系名和片段名

分片的表示方法

■ 分片树表示方法

例:



分配设计

□ 全局数据经过分片设计，得到各个划分的片段，片段到物理场地的存储映射过程称为分配设计。

□ 分配类型

■ 非复制分配

如果每个片段只存储在一个场地上，称为分割式分布，对应的分布库，称为**全分割式分布库**。

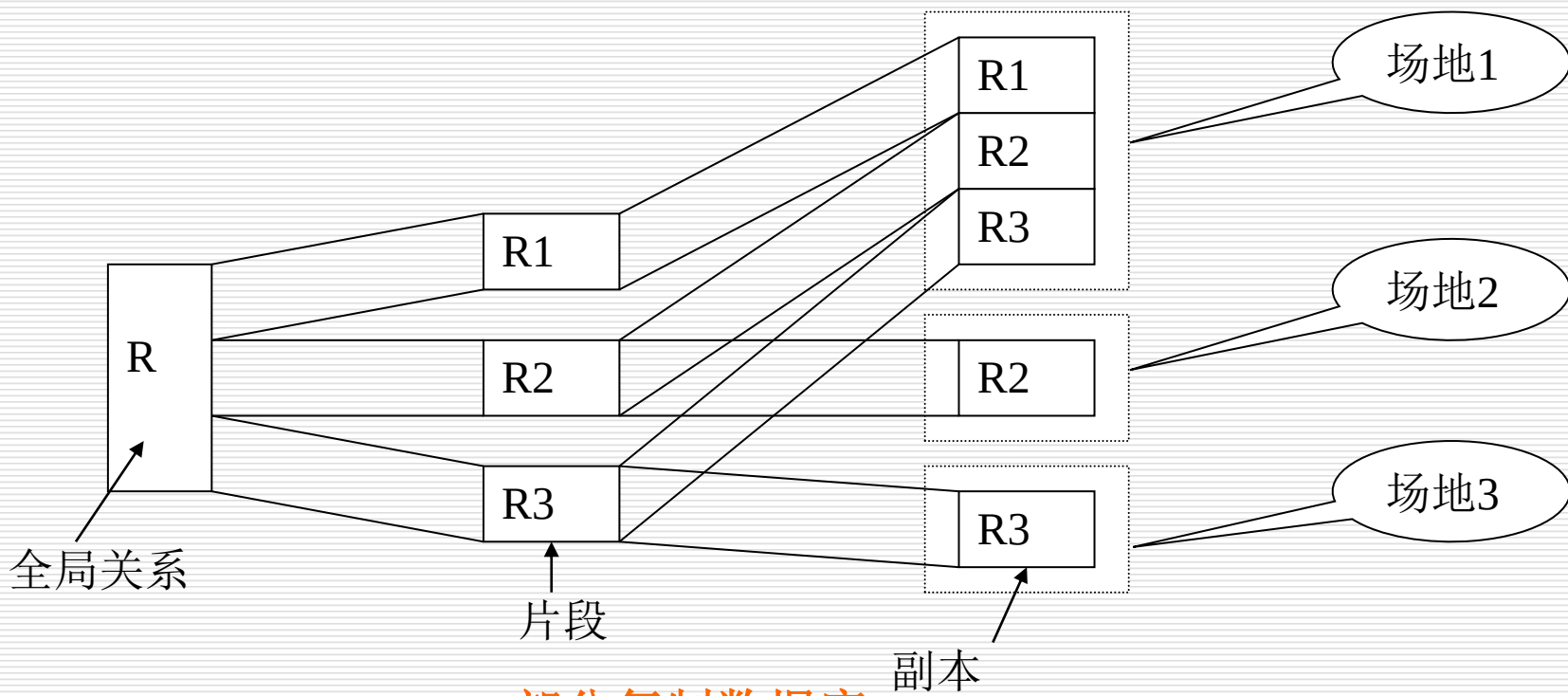
■ 复制分配

- ✓ 如果每个片段在每个场地上存有副本，称为**全复制分配**，对应的分布库称为**全复制分布库**。
- ✓ 如果每个片段只在部分场地上存有副本，称为**部分复制分配**，对应的分布库称为**部分复制分布库**。

分配设计

□ 例：设 R 为全局关系， $R1$ 、 $R2$ 、 $R3$ 为划分的片段。

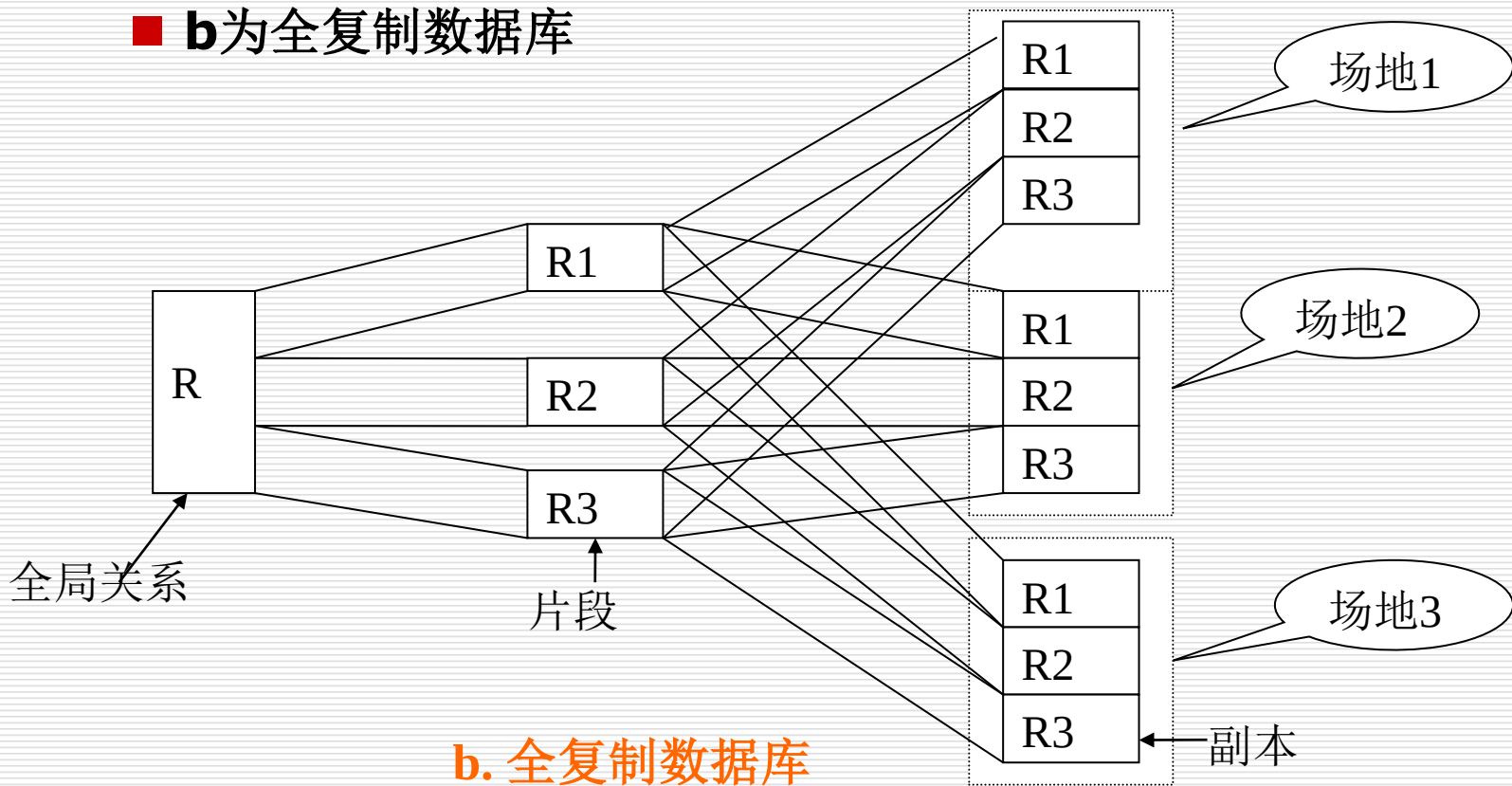
■ a 为部分复制数据库



a. 部分复制数据库

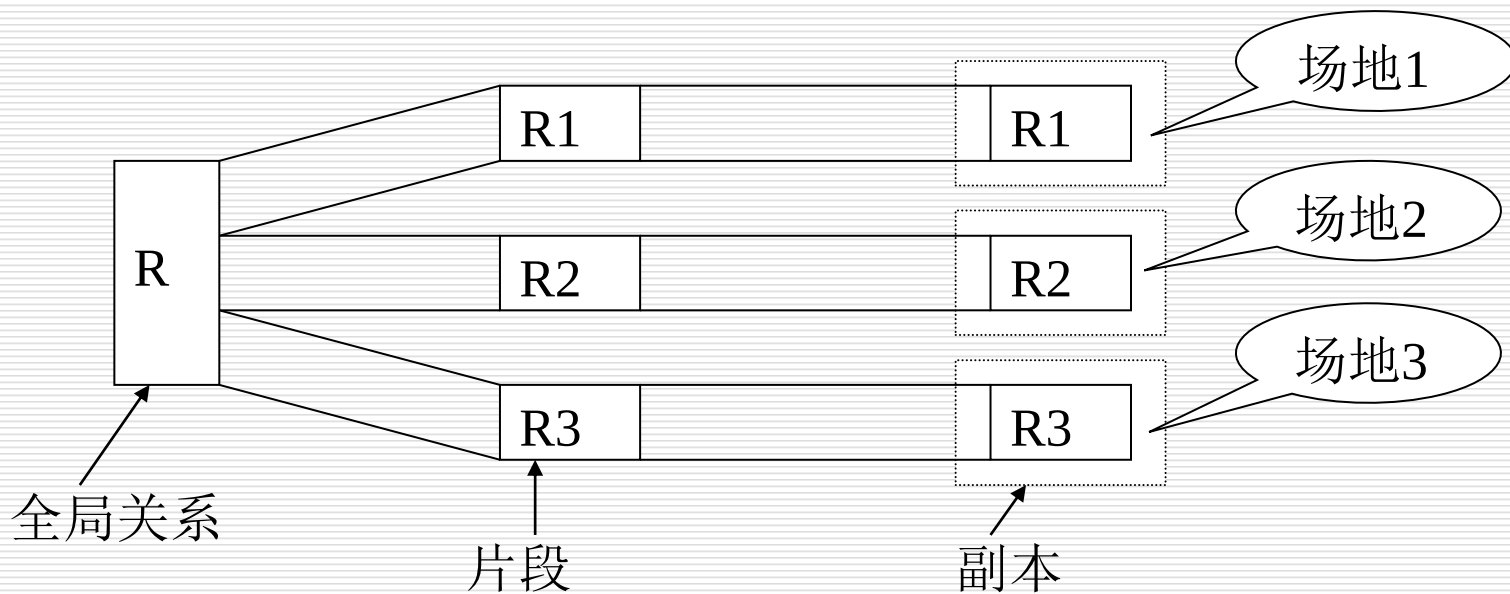
分配设计

- 例：设 R 为全局关系， $R1$ 、 $R2$ 、 $R3$ 为划分的片段。
■ b 为全复制数据库



分配设计

- 例：设 **R** 为全局关系，**R1**、**R2**、**R3** 为划分的片段。
 - **c** 为全分割数据库



c. 全分割数据库

分配设计

□ 分配类型

系统是采用**全分割数据库**或**全复制数据库**或**部分复制数据库**，需根据应用需求及系统运行效率等因素来综合考虑。

一般，从应用角度出发，须考虑：

- 增加事务处理的局部性；
- 提高系统的可靠性和可用性；
- 增加系统的并行性。

从系统角度出发，须考虑：

- 降低系统的运行和维护开销；
- 使系统负载均衡；
- 方便一致性维护。

分配设计

- 从上面几点考虑因素，可知：
- 采用数据复制式分配可增加事务处理的局部性、提高系统的可靠性和可用性，但增加了系统运行的维护开销和系统一致性维护的复杂性；
- 采用全数据分割式分配可增加系统的并行性并可使系统负载均衡，但降低了事务处理的局部性和系统的可靠性和可用性。
- 如何进行片段的分配需综合考虑应用和系统两方面的需求，以求得到一个最佳的分布式数据库系统。
- 一般，如果存在只读查询/更新查询 $>> 1$ ，
则定义为复制式分配好些。

分配设计

□复制分配/分割分配的优缺点对比见下表:

	全复制	部分复制	全分割
查询处理	易	适中	适中
并发控制	适中	难	易
可靠性	很高	高	低
目录管理	易	适中	适中
可行性	一般	通用	一般

分配设计

□ 分配设计原则

分配设计时应考虑的具体因素。

■ 数据库因素

- ✓ 片段的大小；
- ✓ 查询对片段的选择度（查询结果的大小）。

■ 应用因素

- ✓ 查询对片段的读频度；
- ✓ 查询对片段的更新频度；
- ✓ 更新查询需访问的片段；
- ✓ 每个查询的启动场地。

■ 场地因素

- ✓ 场地上存储数据的单位代价；
- ✓ 场地上处理数据的单位代价。

■ 网络通讯因素

- ✓ 网络带宽及网络延迟；
- ✓ 场地间的通信代价；
- ✓ 传输的数据帧大小。

分配设计

□ 分配模型

分配设计者要综合考虑各种因素，建立多个分配模型，分别估计各种模型的代价，通过对比、分析，得出最佳的分配方案。

■ 总代价 = $\sum S_k + \sum Q_i$

$\sum S_k$ 为所有场地上的片段存储代价

$\sum Q_i$ 为所有场地上的查询处理代价

分配设计

■ **分配模型** - 总代价 = $\sum S_k + \sum Q_i$

说明:

① S_k 为在场地 S_k 上的片段存储代价

$$S_k = \sum F_{jk}$$

F_{jk} 为片段 F_j 在场地 S_k 上的存储代价

$\sum F_{jk}$ 为所有片段在场地 S_k 上的存储代价

$$F_{jk} = C_k * \text{SIZE}(F_j) * X_{jk}$$

C_k 为 S_k 上的单位存储代价

$\text{SIZE}(F_j)$ 为 F_j 的大小

$$X_{jk} = \begin{cases} 1 & \text{若 } F_j \text{ 存储在场地 } S_k \text{ 上} \\ 0 & \text{否则为 } 0 \end{cases}$$

分配设计

■ 分配模型-总代价= $\sum S_k + \sum Q_i$

说明:

② Q_i 为查询*i*的处理代价

$$Q_i = P_i + T_i$$

a. P_i 为CPU处理代价

P_i = 访问代价 (A_i) + 完整性约束代价 (I_i) + 并发控制代价 (L_i)

$A_i = \sum S_k \sum F_j$ (更新访问次数+读访问次数) * 局部处理代价 * X_j

$\sum S_k$ 表示所有场地上

$\sum F_j$ 表示所有片段

b. T_i 网络传输代价

$T_i = T_{ui} + T_{Ri}$; T_{ui} 为更新代价; T_{Ri} 为检索代价

$T_{ui} = \sum S_k$ 更新消息代价+ $\sum S_k$ 应答代价

$T_{Ri} = \sum S_k$ (检索命令消息代价+返回结果代价)

分配设计

■ 分配模型-约束条件

① 响应时间 (Q)

执行 Q 的时间 $\leq \text{MAX}.Q$ 的允许响应时间。

② 存储 (S_k) -----指一个场地上的存储约束

$\sum F_j$ 在场地 S_k 上的存储量 $\leq S_k$ 上的许可存储空间。

③ 处理 (S_k) -----指一个场地上的处理约束

$\sum q_i$ 在场地 S_k 上的处理负载 $\leq S_k$ 上的最大许可负载;

$\sum q_i$ 表示所有查询。

数据复制技术

□ 数据复制的优势

- 减少网络负载。就近访问，减少网络传输代价。
- 提高系统性能。利用本地资源，并行处理。
- 更好地均衡负载。负载分布到多个节点。

□ 采用复制式存储技术也增加了数据一致性的维护代价。如同步数据时，如何有效地解决冲突等。

数据复制技术

□ 数据复制的分类

■ 根据更新传播方式不同，分为同步复制和异步复制

✓ 同步复制方法是指所有场地上的副本总是具有一致性。

优点：同步复制方法的优势是实时保证了副本数据的一致性。

不足：1) 需要场地间频繁通信并及时完成事务操作；2) 导致冲突增加，增加了事务响应时间。

适用场合：对于一致性要求较高的商业应用中。

✓ 异步复制方法是指各场地上的副本不要求实时一致性，允许在一定时间内是不一致的。

优点：降低了通信量和冲突概率，缩短了事务响应时间，提高了系统效率。

不足：系统不能显示实时的结果，同时也存在潜在的数据冲突，增加了事务回滚的代价。

适用场合：侧重提高系统效率的应用，如大用户群实时访问大数据量的查询应用。

数据复制技术

□ 数据复制的分类

- 根据参与复制的节点间的关系不同，分为主从复制和对等复制。

- ✓ 主从复制也称单向复制。主场地上的副本称为主副本，从场地上的副本称为从副本。更新操作只能在主场地上进行，并同步到从场地的从副本上。

分析： 主从复制实现简单，易于维护数据一致性。但由于数据只能在主场地上更新，降低了系统的自治性。

- ✓ 对等式复制也称双向复制。各个场地的地位是平等的，可修改任何副本。被修改的副本临时转换为主副本，其它为从副本。主副本所在场地为协调场地，协调同步所有从场地上的从副本。

分析： 对等式复制中，各场地具有高度自治性，系统可用性好。但由于允许更新任何副本，会引起事务冲突，需要引入有效的冲突解决机制，处理复杂，系统开销大。

数据复制技术

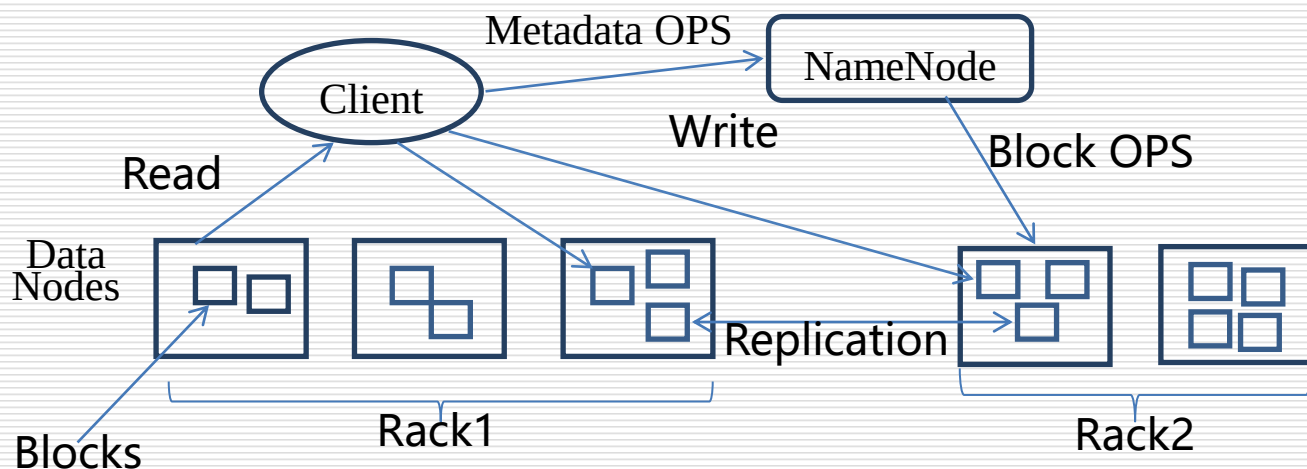
□ 复制的常用方法

- **基于触发器法**：在主场地的主数据表（主副本）中创建相应的触发器，当主表数据进行更新、插入和删除操作并成功提交时，触发器就会被触发，将当前副本的变化反映到从副本中。可用于同步复制，但对于对等式复制和异构复制较难实现。占用的系统资源较多，影响系统运行效率，较适宜于小型数据库应用。
- **基于日志法**：通过分析数据库的操作日志信息来捕获复制对象的变化。当主副本被更改时，复制代理只需将修改日志信息发送到从场地，由从场地代理实现本地数据的同步。实现方便，不占用太多额外的系统资源，对任何类型的复制都适用。
- **基于时间戳法**：主要根据数据的更新时间来判断是否是最新数据。需要为每一个副本数据表定义一个时间戳字段，用于记录每个表的修改时间，并需要监控程序监控时间戳字段的时间。该方法适合于对数据更改较少的系统。
- **基于API法**：在应用程序和数据库之间引入第三方的程序（如中间件），通过**API**完成。在应用程序对数据库修改的同时，记录复制对象的变化。该方法可减轻**DBA**的负担，但无法捕获没有经过**API**的数据的更改操作，具有一定局限性。

大数据数据库的分布存储策略

□ 分布式文件系统-HDFS

- **HDFS:** 分布式文件系统，适合存储超大文件（几百MB、GB甚至TB级别的文件）。将超大文件分割成多个块（block），block大小默认为64MB，每一个block会在多个数据节点（DataNode）上存储多份副本，默认是3份。HDFS采用一次写入、多次读取的流式数据访问模式。



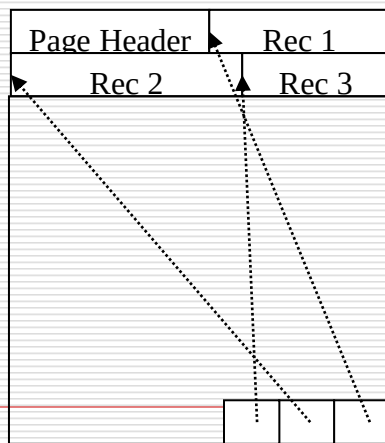
HDFS基本架构图

大数据数据库的分布存储策略

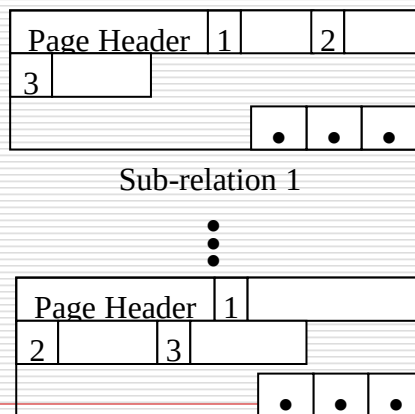
□ 分布式文件系统-HDFS

■ 数据在HDFS文件块中的存储模式

- **NSM**(N-ary Storage Model)存储模式：主要由页头（Page header）、数据（Body）、以及每行数据（或称每条记录）在当前page中位置的偏移量（Trailer）组成。**记录数据**从每一磁盘页的开始**连续存放**，在页的尾端存放记录的偏移地址（offset），用于定位每一记录的开始。

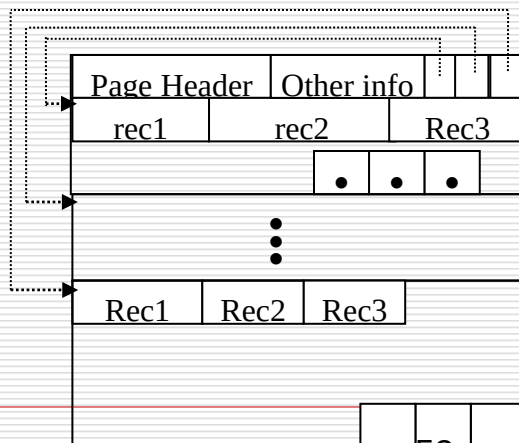


(a)NSM



Sub-relation 1
⋮
Sub-relation n

(b)DSM



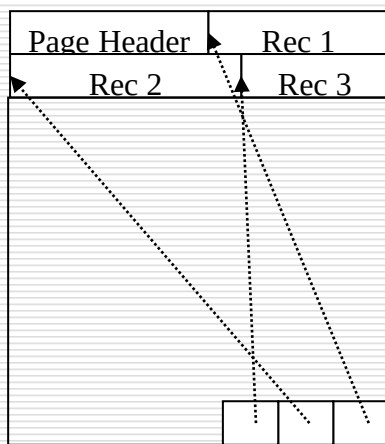
(c)PAX

大数据数据库的分布存储策略

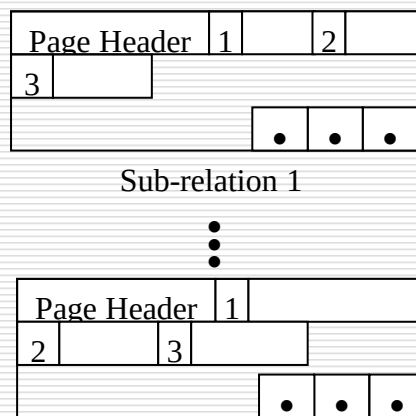
□ 分布式文件系统-HDFS

■ 数据在HDFS文件块中的存储模式

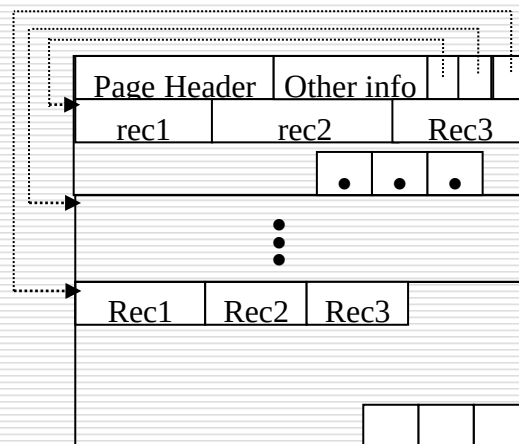
- **DMS** (Decomposition Storage Model)存储模式：是将关系**垂直划分**为多个子关系，可细分为全DSM（full DSM）模式和部分DSM（partial DSM）模式



(a)NSM



Sub-relation n

(b)DSM

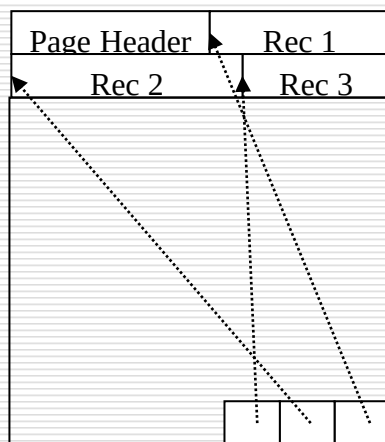
(c)PAX

大数据数据库的分布存储策略

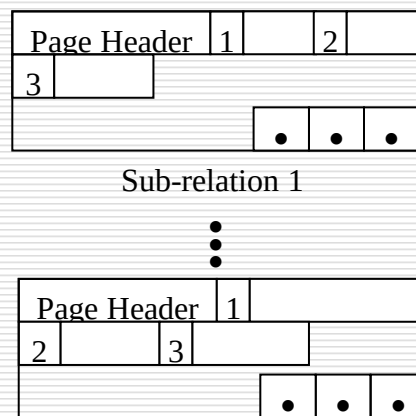
□ 分布式文件系统-HDFS

■ 数据在HDFS文件块中的存储模式

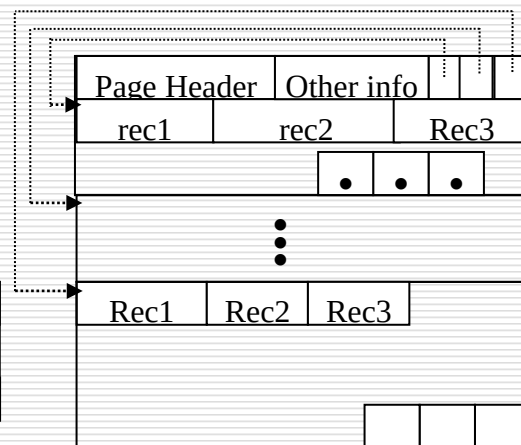
- **PAX** (Partition Attributes Across)存储模式：针对每一页内的记录的进一步垂直分区的模式，即按同一属性值再分组为迷你页（minipage）



(a)NSM



Sub-relation n
(b)DSM

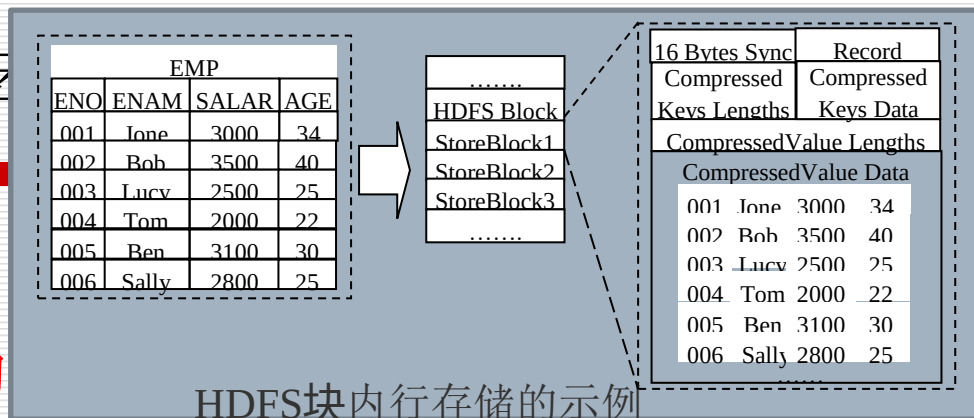


(c)PAX

大数据数据库的分布存

□ 分布式文件系统-HDFS

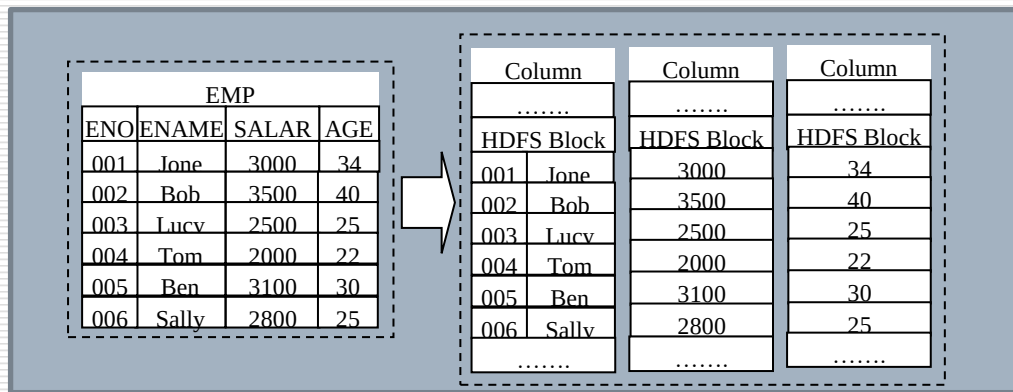
■ HDFS块中数据存储结构



□ 行存储模式：基于NSM模式

□ 列存储模式：基于DSM存储模式

□ 混合存储模式：基于PAX存储模式的混合存储方式



HDFS块内列存储的示例

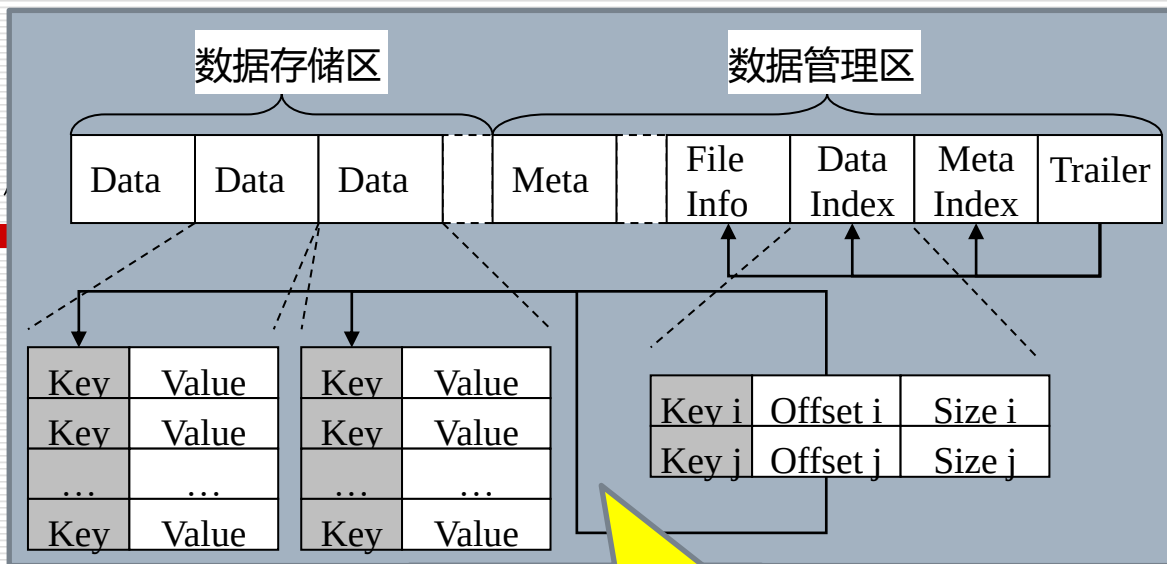
大数据数据库的分布存储策略

□ 基于**SSTable**的数据存储结构

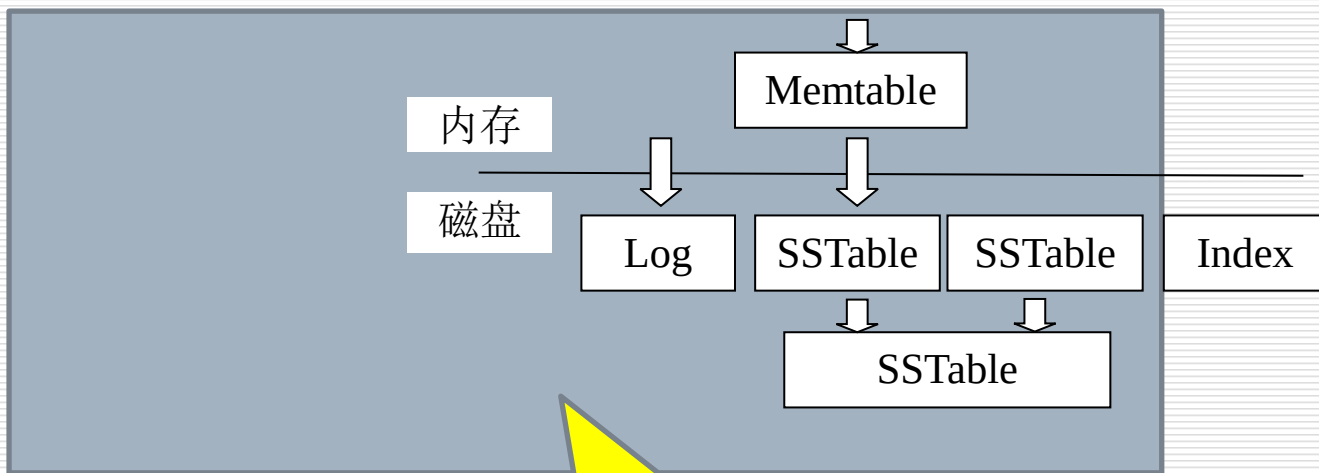
■ 基本的**SSTable**数据存储结构

- SSTable (Static Search Table) : 存储数据的文件格式, 其内部提供了一个一致性的、有序的从“键”(Key)到“值”(value)的不可变映射(MAP)。
- 每个SSTable内部包含一系列的**数据块**(通常每个块是64KB大小, 且其大小是可配置的)和一个**数据块索引**(保存在SSTable的尾部), 用来定位数据块, 当SSTable打开时该索引会被加载到内存。

大数据



数据存储结构



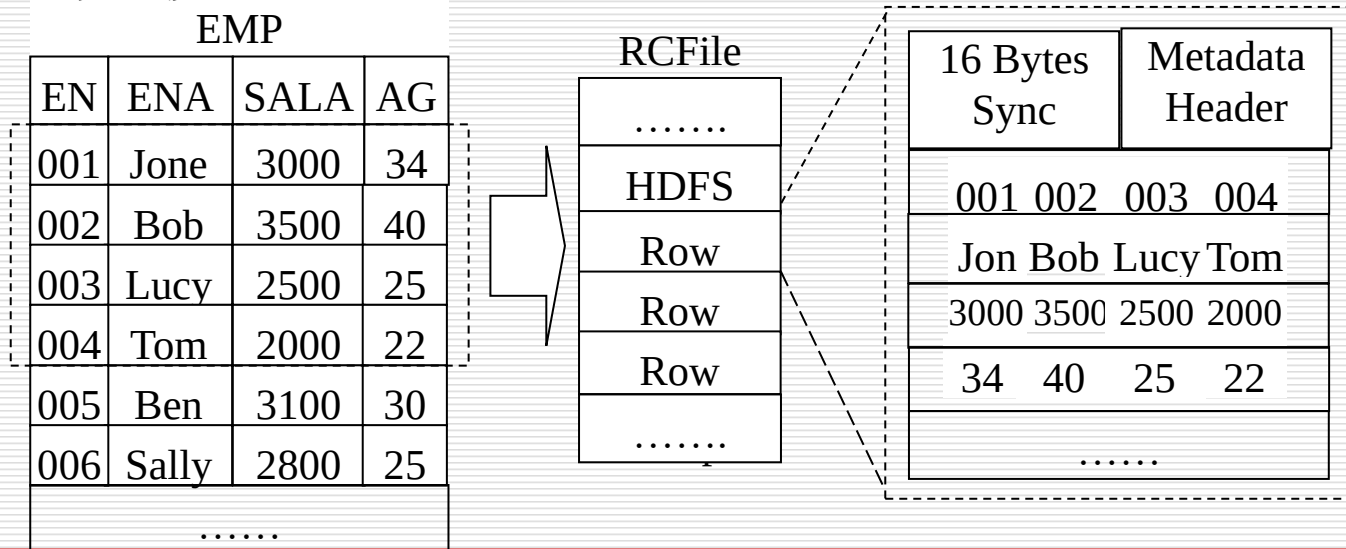
文件组织结构

大数据数据库的分布存储策略

□ 基于SSTable的数据存储结构

■ 基于SSTable的优化存储结构-RCFile

RCFile (Record Columnar File) 是一种高效的数据存储结构，具有如下特点：1) 快速数据装载；2) 快速查询处理；3) 高效的存储空间利用率；4) 动态负载模式的强适应性。



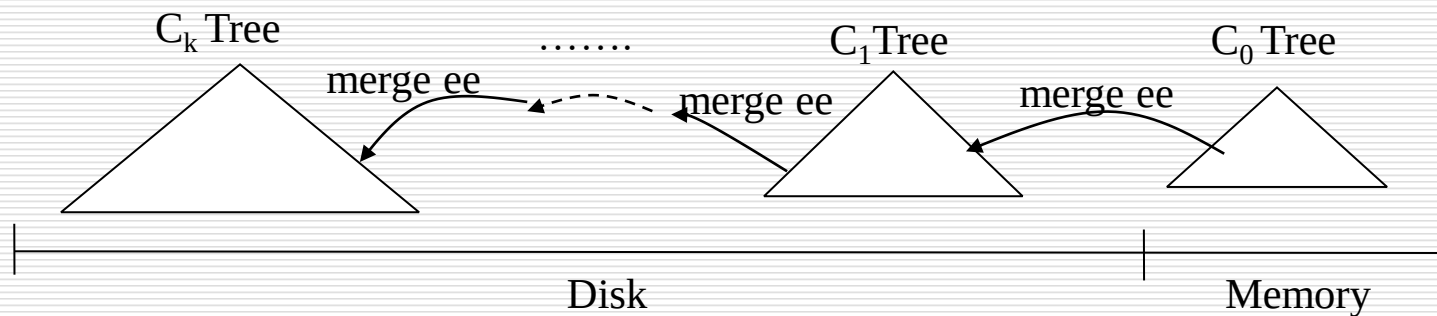
RCFile在HDFS块内行列混合存储的示例

大数据数据库的分布存储策略

□ 分布式文件系统-HDFS

■ LSM-Tree 存储结构

- 对数据的增量更新暂时保存在内存中，达到指定的存储阈值后将该批更新批量写入到磁盘
- 在批量写入的过程中，同已经存在的数据做合并操作
- 在读取数据时，同样需要合并磁盘中数据和内存中最近修改操作



LSM-Tree多层示意图



文档模型

大数据数据库的分布存储策略

□ 大数据存储模型

□ 关系模型：行存储模型；列存储模型

□ 键值模型

□ 文档模型

```
{id: 0000000000000001,
ENO: '002',
Work:{
ENAME: 'Bob',
SALARY: 3500,
DNO: '103'
},
Contact:{
Address: 'XXX, Road',
Tel: '11002233',
Email: 'bob@xx.com'
}
}
```

Key 1	Value 1: Byte[]
Key 2	Value 2: Byte[]
...	
...	
Key n	Value n: Byte[]

键值模型

Column Family: EMP

Key(ENO)	Columns		
002	Column Key	Column Value	Timestamp
	"ENAME"	"Bob"	4
	"SALARY"	"3500"	4
	"DNO"	"103"	4
003	Column Key	Column Value	Timestamp
	"ENAME"	"Lucy"	6
	"SALARY"	"2500"	6
	"DNO"	"102"	6
	"TEL"	"81455588"	6

Column Family: EMP			
Key(ENO)	Super Columns		
002	Key	Columns	
		Column Key	Column Value
		Timestamp	
		Column Key	Column Value
	"WORK"	"ENAME"	"Bob"
		"SALARY"	"3500"
		"DNO"	"103"
	"Contact"	Column Key	Column Value
		Timestamp	
		Column Key	Column Value
		Timestamp	
		"Address"	"XXX, Road"
		"Tel"	"11002233"
		"Email"	"bob@xx.com"

超级列族模型

列族模型

大数据数据库的分布存储策略

□ 数据分区策略

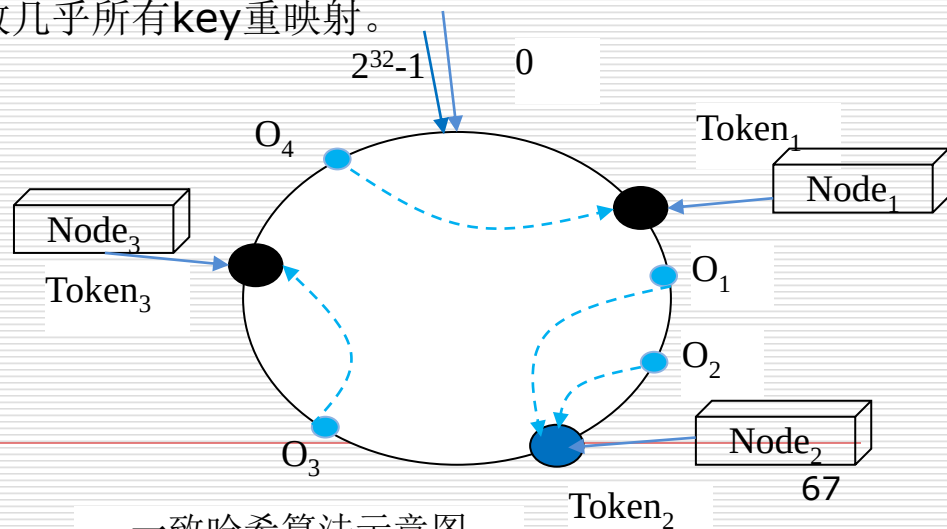
□ 范围分区

- 范围分区是指依据分区键值划分每个分区，如典型按照时间范围来分区。
- 通常存在一份全局元数据，用于记录数据的分区划分情况以及分区所存在的位置，可以以服务器为单位描述服务器所负责管理的数据分区。

□ 一致性哈希分区（Consistent hashing）

- 为了解决因特网中的热点(Hot spot)问题，是一种特殊类型的哈希。当调整哈希表大小时（如hash节点故障时），一致性哈希平均只有 k/n 个键需要重映射（ k 是key的数量， n 是数据槽（slots）的数量）。而传统的哈希表，一个数据槽的变化将导致几乎所有key重映射。

每个节点实际的负责范围从简单哈希函数的单个值变成了一个值区间。

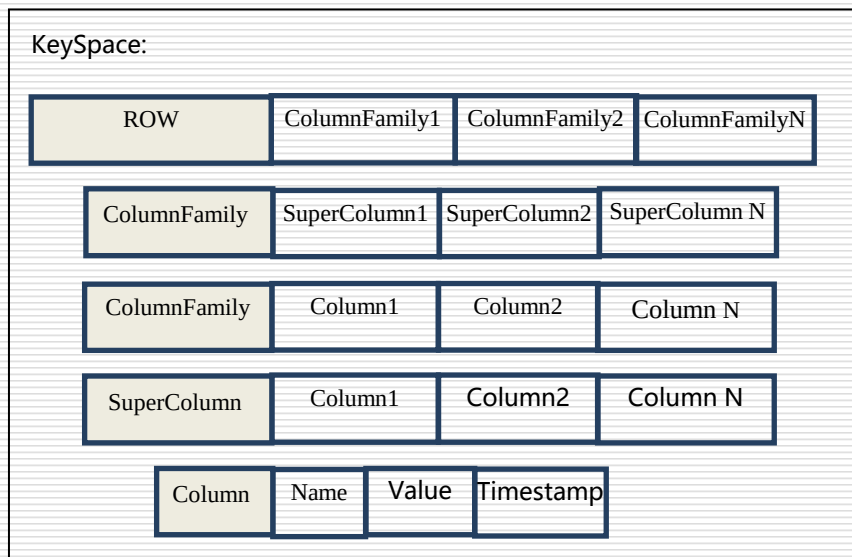


大数据数据库分布存储样例

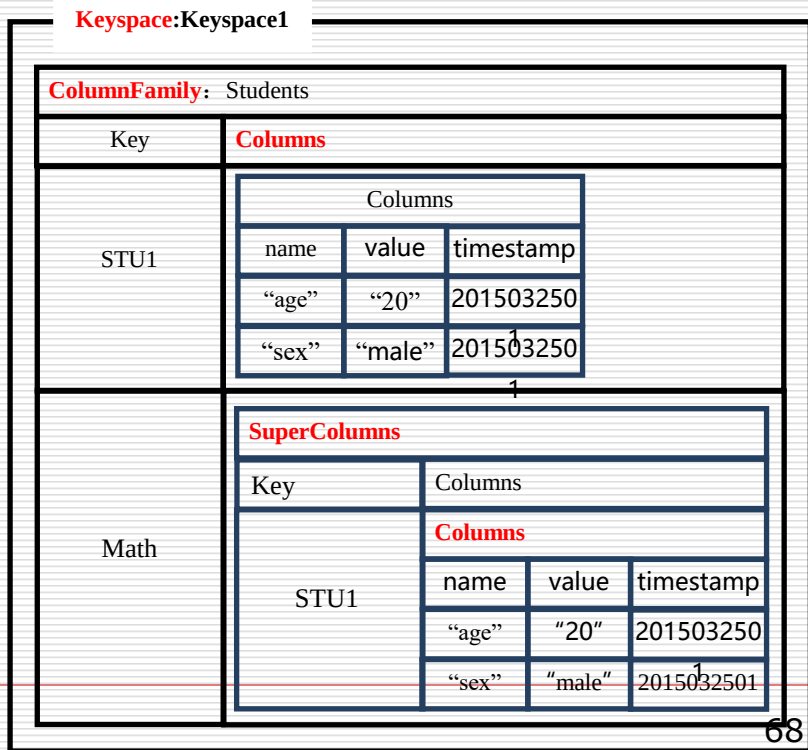
□ Cassandra

□ 数据模型

□ Keyspace是数据最外层， ColumnFamily都属于某一个Keyspace



Cassandra数据模型图



Cassandra数据模型组织示例图

大数据数据库分布存储样例

□ Cassandra

□ 数据分布策略

□ 分布哈希存储策略

□ 分区 (Partitioner) 类型

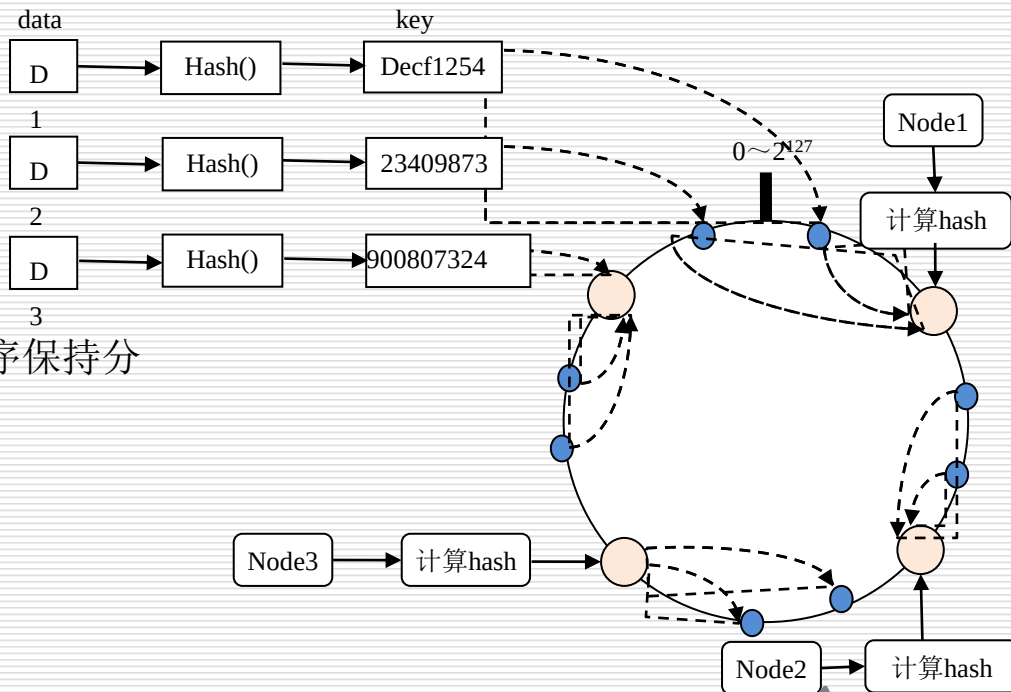
- 随机分区
- 顺序保持分区
- 校对顺序保持分区 (同顺序保持分区不同是排序方式不一样)

□ 副本配置策略

- 机架未知策略
- 机架感知策略
- 共享数据中心策略

□ 存储机制

- 基于列族数据模型
- 采用 Memtable 和 SSTable 的方式进行存储。



基于一致性哈希的数据存放策略



第三章 结束

